Generalización de Código de Automatización en Desarrollo de Software con IA

Tema 1. Aprendizaje automático aplicado a la generación de código

Índice

Esquema

Ideas clave

1.1. Introducción y objetivos

1.2. Aplicaciones del aprendizaje automático en generación de código

1.3. Selección y preprocesamiento de datos para entrenamiento

1.4. Modelos de lenguaje especializados en programación

1.5. Evaluación de modelos de generación de código

1.6. Casos de uso prácticos en desarrollo de software

1.7. Referencias bibliográficas

A fondo

GitHub Copilot: the AI-powered code completion tool

OpenAI Codex: evaluating large language models trained on code

Machine learning for code generation (vídeo)

Aroma: using machine learning for code recommendation

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

1.1. Introducción y objetivos

La evolución de la **inteligencia artificial** (IA) permitió avances significativos en diversos campos, incluido el desarrollo de *software.* Uno de los ámbitos más prometedores es la **generación de código mediante aprendizaje automático** (*machine learning* o ML), ya que permite automatizar tareas repetitivas, mejorar la eficiencia y reducir errores. Esta disciplina, que aplica modelos de IA para analizar grandes volúmenes de datos y aprender patrones, está transformando la manera en que los desarrolladores **crean, depuran y mantienen los códigos.**

A través de algoritmos avanzados, como las redes neuronales profundas y los modelos de lenguaje transformador (como GPT-3 o Codex), es posible **automatizar tareas** que tradicionalmente requerían intervención manual. Estos modelos pueden generar código a partir de **descripciones en lenguaje natural,** sugerir fragmentos de código durante la escritura, corregir errores en tiempo real y hasta refactorizar código existente para optimizarlo. Todo esto abrió nuevas posibilidades para **mejorar la productividad** de los desarrolladores, permitir prototipos rápidos y, en algunos casos, desarrollar aplicaciones completas con mínima intervención humana.

Sin embargo, la automatización de la generación de código no es un proceso exento de desafíos. La **calidad del código** generado, su capacidad para integrarse en sistemas complejos y la **supervisión humana requerida** para asegurar su funcionalidad adecuada son algunas de las preocupaciones actuales. A medida que los modelos de IA continúan mejorando, es probable que estas herramientas no solo apoyen, sino que **reemplacen** en ciertos casos tareas rutinarias de codificación.

De esta manera, esto permite que los desarrolladores puedan centrarse en aspectos más creativos y de mayor valor agregado.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Se espera que, en el futuro cercano, estas herramientas se integren aún más con **flujos de trabajo de DevOps,** donde la generación de código, las pruebas y los despliegues se realizan de manera **automática y continua.** También se vislumbra un impacto en la **modernización de sistemas heredados,** lo que permite que la IA automatice la traducción de código de lenguajes obsoletos a lenguajes modernos.

Los objetivos del tema son:

- Entender los principios básicos del aprendizaje automático aplicados a la generación de código.
- Conocer las herramientas y los modelos de IA más utilizados en la automatización de la escritura de código.
- Aplicar técnicas para la selección y el preprocesamiento de datos de código, esenciales para el entrenamiento de modelos de IA.
- Evaluar el rendimiento de los modelos generativos de código, identificando sus fortalezas y limitaciones.
- Analizar casos de uso prácticos de generación automática de código y comprender su impacto en la productividad y calidad del software.
- Desarrollar competencias para integrar modelos generativos en proyectos de *software* y para automatizar tareas repetitivas.

© Universidad Internacional de La Rioja (UNIR)

1.2. Aplicaciones del aprendizaje automático en generación de código

El aprendizaje automático por *machine learning* (ML) abrió un nuevo horizonte en el campo del **desarrollo de *software,*** especialmente en la automatización y generación de código. El uso de algoritmos de ML les permitió a los sistemas no solo **analizar patrones** en grandes volúmenes de datos, sino también **generar y corregir código** de manera eficiente.

El aprendizaje automático (AA) tiene aplicaciones fundamentales en la **generación de código,** que van desde la creación de pequeñas funciones hasta sistemas completos. Por ejemplo, hoy en día, plataformas como GitHub Copilot utilizan Codex, un modelo de **lenguaje especializado en programación,** para generar código a partir de simples descripciones en **lenguaje natural.**

Sin embargo, el AA va más allá, dado que también permite la **generación automática de microservicios.** Estos modelos son capaces de **generar, probar e integrar** automáticamente código en *pipelines* de CI (integración continua) y CD (distribución continua), por lo que optimizan el flujo de desarrollo en empresas grandes.

Otra área de impacto es la **modernización de *software* legado,** donde las IA pueden refactorizar automáticamente código antiguo o convertir aplicaciones basadas en lenguajes obsoletos a **lenguajes actuales,** como Python o JavaScript.

Aunque estas herramientas pueden mejorar la productividad, todavía requieren supervisión humana para asegurar la calidad y la precisión del código generado.

© Universidad Internacional de La Rioja (UNIR)

A medida que los modelos continúan mejorando, se espera que estas herramientas se conviertan en **asistentes cada vez más poderosos,** capaces de comprender contextos más amplios y de realizar tareas de programación más complejas de forma autónoma. A continuación, se describen las **principales aplicaciones del AA** en la generación de código.

Generación de código a partir de lenguaje natural

Una de las aplicaciones más significativas del AA es la capacidad de generar código a partir de descripciones en lenguaje natural. Los modelos de lenguaje basados en **redes neuronales profundas,** como GPT-3 y Codex, pueden procesar texto en lenguaje natural y traducirlo en código en **diversos lenguajes de programación.** Esta tecnología es fundamental para herramientas como GitHub Copilot, que sugiere fragmentos de código basados en comentarios o descripciones de alto nivel proporcionadas por el programador (Chen et al., 2021).

Este tipo de aplicaciones es particularmente útil en la creación de prototipos rápidos o como soporte para desarrolladores en la implementación de fragmentos de código repetitivos.

La base de esta aplicación radica en el uso de **redes neuronales recurrentes y *transformers,*** entrenados en grandes *corpus* de datos que incluyen documentación y código fuente de proyectos de *software* de diferentes lenguajes. Estos modelos son capaces de **capturar dependencias y estructuras complejas,** lo que les permite generar un **código preciso y funcional.**

La Figura 1 muestra cómo el modelo procesa una entrada en lenguaje natural y produce código, de manera que pasa por las fases de **preprocesamiento, codificación y decodificación** (entrada, representación y generación de código).

© Universidad Internacional de La Rioja (UNIR)

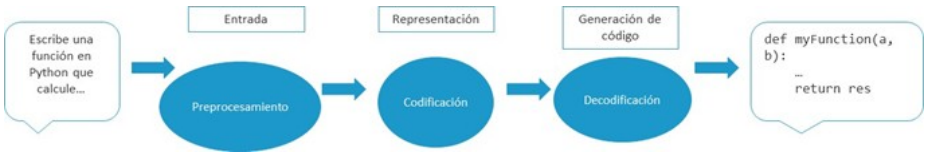


Figura 1. Procesamiento de una entrada en lenguaje natural a código. Fuente: elaboración propia.

Supongamos que se le pide a un modelo generar una función en Python que calcule el factorial de un número. La instrucción proporcionada sería algo como «Escribe una función en Python que calcule el factorial de un número». Un modelo como Codex de OpenAI (un derivado de GPT-3) puede generar automáticamente el siguiente código:

```
def factorial(n):

if n == 0:

return 1

else:

return n * factorial(n-1)
```

Autocompletado y sugerencias de código

El AA también transformó la forma en que se sugiere y autocompleta el código. Herramientas como TabNine y los **sistemas de autocompletado avanzado en IDE** (entorno de desarrollo integrado) modernos utilizan modelos predictivos entrenados en grandes bases de datos de código para **sugerir fragmentos, funciones o incluso líneas completas de código.** Estos sistemas mejoran con el tiempo, ya que aprenden del estilo y las preferencias de cada programador, lo que los convierte en asistentes personalizados y eficaces (Xu et al., 2022).

© Universidad Internacional de La Rioja (UNIR)

Este tipo de asistencia permite un desarrollo más rápido y eficiente, de manera que alivia la carga cognitiva de recordar detalles específicos de la sintaxis o lógica de programación.

Además de mejorar la productividad, este tipo de tecnología contribuye a **reducir errores comunes** y a **agilizar el proceso de desarrollo,** lo que permite que los desarrolladores se concentren en aspectos más creativos y complejos del código.

> Si un desarrollador comienza a escribir una función que maneja archivos en Python (`with open('archivo.txt', 'r') as f:`), el modelo puede sugerir automáticamente el bloque de código completo para leer las líneas del archivo:
>
> ```
> lines = f.readlines()
>
> for line in lines:
>
> print(line.strip())
> ```

Corrección automática de código

Otra aplicación relevante es la corrección automática de errores de código (autocorrección), que puede **identificar defectos o errores de sintaxis** en tiempo real mientras el programador está escribiendo. Algunos modelos de lenguaje, como Codex y otros enfoques basados en aprendizaje profundo, son capaces de detectar **patrones en el código fuente** que suelen derivar en errores y proponer soluciones correctivas de manera automática (Ray et al., 2016).

© Universidad Internacional de La Rioja (UNIR)

Esto es especialmente relevante en entornos de desarrollo donde el **tiempo es crítico** y los errores humanos son inevitables. Este tipo de aplicaciones no solo mejoran la calidad del código, sino que también **reducen el tiempo de depuración** y disminuyen la probabilidad de que se produzcan **errores críticos en fases posteriores** del desarrollo.

> Si un desarrollador escribe un bucle mal estructurado:
>
> ```
> for i in range(10):
>
> print i
> ```
>
> Un modelo puede identificar que falta el uso de paréntesis en la función `print` y sugerir la corrección:
>
> ```
> for i in range(10):
>
> print(i)
> ```

Optimización y refactorización automática de código

El AA también se utiliza para realizar la **refactorización automática de código,** una técnica que reestructura el código existente para **mejorar su eficiencia y mantenibilidad** sin cambiar su funcionalidad externa. Estas herramientas son especialmente útiles en contextos de **alto rendimiento** o en sistemas con restricciones de recursos, como los dispositivos móviles.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

A través del análisis de múltiples versiones de un programa, los modelos pueden identificar cómo mejorar el rendimiento o reducir el consumo de recursos.

Modelos entrenados en grandes volúmenes de código pueden **identificar fragmentos que podrían optimizarse,** así como pueden sugerir modificaciones que mejoran la legibilidad, reducen la complejidad y **eliminan redundancias.** Este tipo de tecnologías es clave para **facilitar el mantenimiento** de grandes bases de código, especialmente en proyectos de larga duración (Allamanis et al., 2018).

Generación de pruebas automatizadas

El AA se utiliza para generar automáticamente casos de prueba para código recién escrito o existente. Los modelos pueden analizar el código fuente y generar conjuntos de pruebas que **verifiquen la funcionalidad de los programas** en distintos escenarios, para reducir la carga de trabajo asociada con la creación manual de pruebas. Este tipo de aplicaciones es particularmente útil en sistemas complejos y proyectos de *software* a gran escala, donde la **cobertura de pruebas** es fundamental para **garantizar la calidad del producto final** (Fraser y Arcuri, 2013).

En algunos casos, la generación de pruebas automáticas se basa en **análisis estáticos o dinámicos** del código existente. Por ejemplo, un modelo podría analizar una clase compleja y generar automáticamente pruebas unitarias que cubran los métodos clave y las interacciones entre ellos.

La Figura 2 muestra el proceso de cómo se genera un caso de prueba basado en la lógica del código, desde la identificación de entradas hasta la validación de las salidas.

© Universidad Internacional de La Rioja (UNIR)

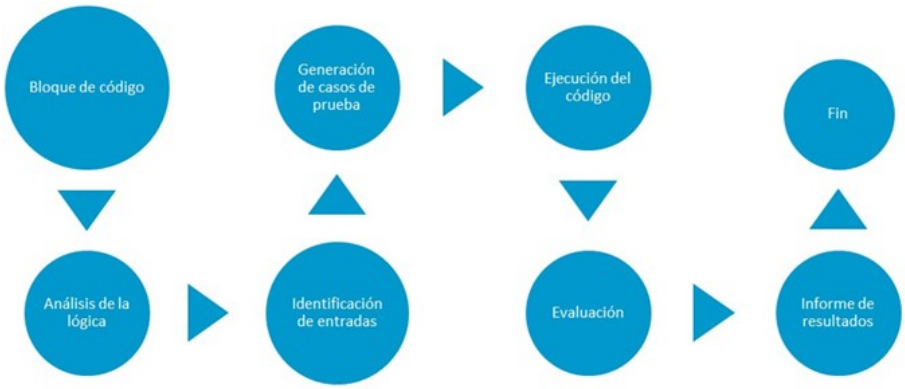


Figura 2. Generación de casos de prueba. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Supongamos que se tiene una función que calcula el área de un círculo en Python:

```
def calcular_area_circulo(radio):

return 3.1416 * radio * radio
```

Un modelo de IA puede generar automáticamente casos de prueba unitarios para verificar la funcionalidad de esta función:

```
import unittest

class TestCalcularAreaCirculo(unittest.TestCase):

self.assertEqual(calcular_area_circulo(1), 3.1416)

self.assertEqual(calcular_area_circulo(2), 12.5664)



def test_area_cero(self):

self.assertEqual(calcular_area_circulo(0), 0)



def test_area_negativa(self):

self.assertEqual(calcular_area_circulo(-1), 3.1416)



if __name__ == '__main__':

  unittest.main()
```


© Universidad Internacional de La Rioja (UNIR)

> En este caso, el modelo genera automáticamente una prueba unitaria para verificar que la función calcule correctamente el área para diferentes valores de entrada, incluidas situaciones como un radio negativo o cero.

Traducción entre lenguajes de programación

Los modelos de AA también son capaces de realizar **traducción de código** entre diferentes lenguajes de programación. Herramientas como TransCoder emplean redes neuronales para convertir código de un lenguaje a otro, aunque mantiene la **lógica y funcionalidad originales.** Esto resulta especialmente útil cuando se migra un *software* legado o cuando se necesitan **traducir aplicaciones** entre diferentes ecosistemas de desarrollo (Rozière et al., 2020).

© Universidad Internacional de La Rioja (UNIR)

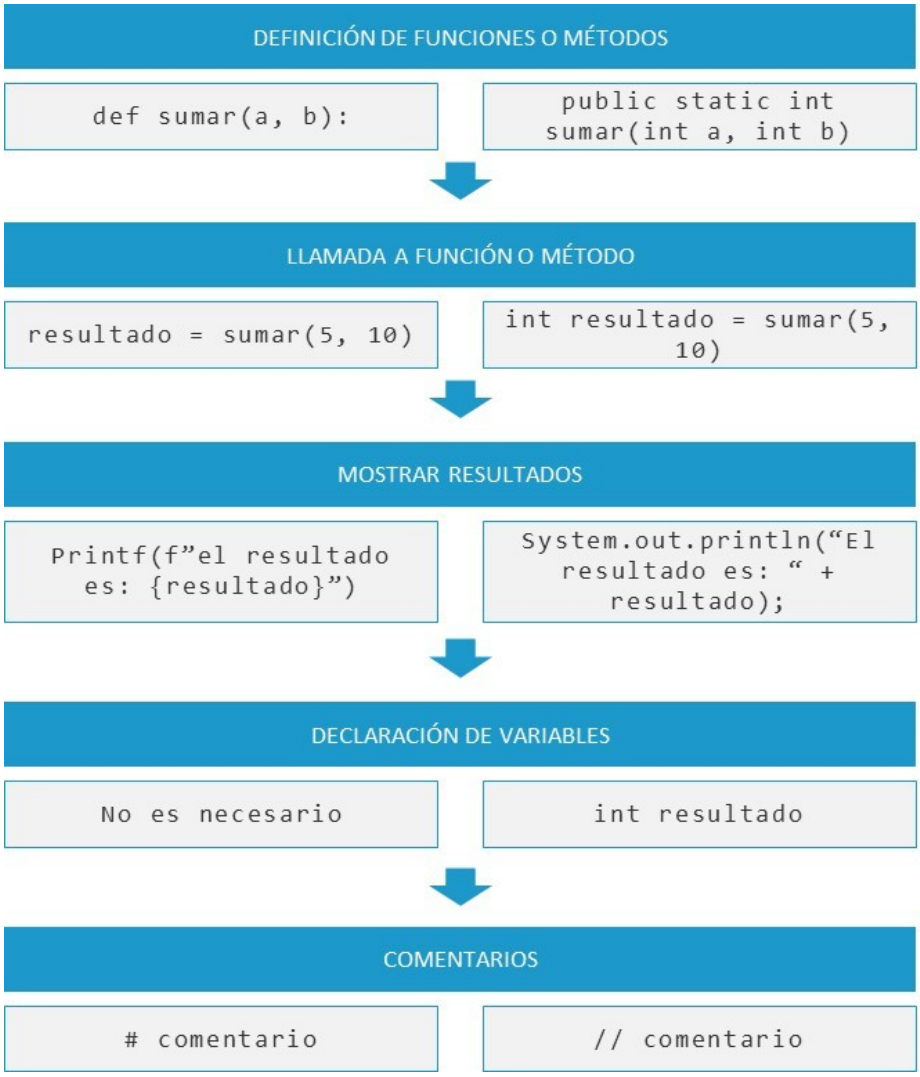


Figura 3. Correspondencias entre Python y Java. Fuente: elaboración propia.

La Figura 3 muestra las **correspondencias de sintaxis** de un fragmento de código en Python que se traduce a su equivalente en Java, por lo que se destacan las diferencias clave en estructura y tipos de datos.

La traducción automática entre lenguajes de programación puede verse limitada por las dependencias de librerías externas.

© Universidad Internacional de La Rioja (UNIR)

Supongamos que tenemos la siguiente función en Python que calcula el máximo común divisor (MCD) a través del algoritmo de Euclides:

```
def mcd(a, b):

while b != 0:

a, b = b, a % b

return a
```

Un modelo de traducción automática podría generar su equivalente en Java:

```
public class Calculadora {

public static int mcd(int a, int b) {

while (b != 0) {

int temp = b;

b = a % b;

a = temp;

}

return a;

}

}
```


© Universidad Internacional de La Rioja (UNIR)

Aquí, el código generado mantiene la misma lógica, pero adapta la sintaxis y los tipos de datos a las convenciones de Java. Este tipo de traducción puede **facilitar la migración de proyectos,** de manera que permite que un programa escrito en un lenguaje sea **compatible con otra plataforma.**

Generación automática de documentación

Otro uso del AA en la generación de código es la creación automática de documentación a partir de código fuente. Los modelos pueden **analizar el código** y **generar comentarios** que expliquen su funcionamiento, así como crear **documentación más formal.** Por ejemplo, a partir de una función compleja, un modelo podría generar automáticamente un comentario que explique su propósito y cómo utilizarla, lo que mejora la legibilidad del código y facilita su mantenimiento.

La Figura 4 muestra un bloque de código antes y después de ser documentado automáticamente por el modelo.

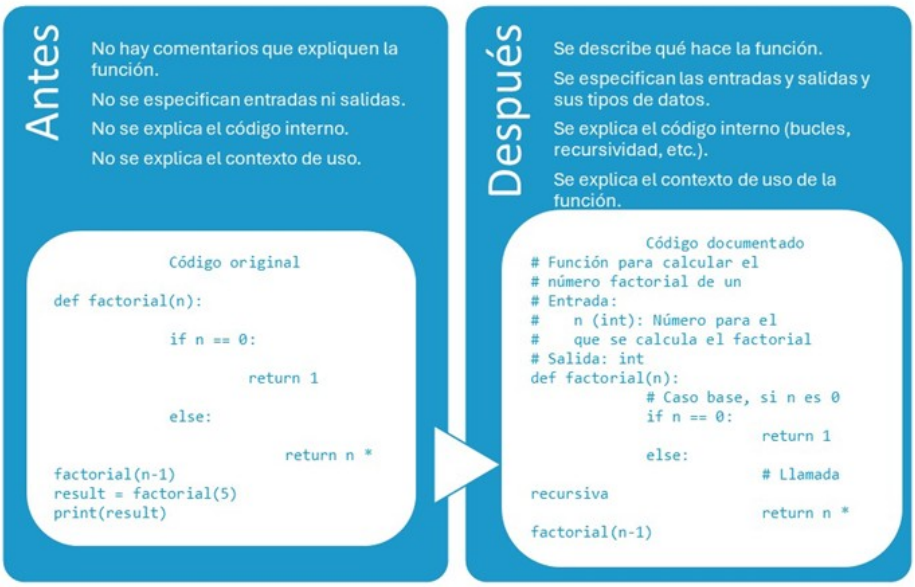


Figura 4. Documentación automática de código. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

1.3. Selección y preprocesamiento de datos para entrenamiento

Los modelos generativos de código, como los basados en *transformers* (Codex, AlphaCode, etc.), requieren **grandes volúmenes de datos de código** para su entrenamiento. Estos datos deben seleccionarse cuidadosamente y procesarse de manera adecuada para garantizar que el **modelo aprenda correctamente** los patrones de programación y las convenciones de los lenguajes de código.

El preprocesamiento de datos es un paso crítico en la creación de modelos de IA que **generen código de alta calidad,** lo que incluye la selección de datos de código fuente, la eliminación de comentarios innecesarios y la **normalización de estilos** de programación. A nivel más avanzado, se pueden realizar técnicas como la **eliminación de dependencias innecesarias,** la normalización de nombres de variables y la *tokenización* específica de lenguajes de programación.

A medida que los modelos de lenguaje siguen evolucionando, la **optimización de estos procesos** será clave para su desempeño en aplicaciones reales. Las empresas que utilizan IA para generación de código suelen implementar *pipelines* complejos para la **recolección de datos** de repositorios de código abierto y de sus propios sistemas internos. Por ejemplo, Google entrenó modelos con vastas cantidades de datos de código mediante la limpieza y la categorización de proyectos que abarcan diferentes lenguajes de programación.

A través de una cuidadosa recolección, limpieza, *tokenización* y normalización, los datos se transforman en un formato adecuado para que el modelo pueda aprender a generar un código eficiente y de calidad.

© Universidad Internacional de La Rioja (UNIR)

Selección de datos para entrenamiento

La selección de datos es crucial para la **efectividad del modelo.** El objetivo es recopilar un **conjunto de datos representativo** que incluya un código funcional, bien escrito, y que abarque múltiples lenguajes de programación y paradigmas. Existen varias fuentes comunes de datos:

- **Repositorios de código abierto.** Los repositorios como GitHub, GitLab y Bitbucket proporcionan grandes cantidades de código fuente en varios lenguajes de programación.
  - Los modelos como Codex se entrenan en miles de millones de líneas de código provenientes de estos repositorios, lo que permite capturar una amplia gama de estilos y enfoques de codificación.
  - GitHub cuenta con más de cien millones de repositorios públicos, lo que lo convierte en una de las fuentes de datos más ricas para la generación de código. Sin embargo, no todo este código es adecuado para el entrenamiento sin filtración.
- **Documentación y ejemplos.** Además de código real, la documentación técnica y los ejemplos de código que aparecen en sitios como Stack Overflow y blogs de desarrolladores pueden enriquecer los datos de entrenamiento. Esto ayuda al modelo a aprender no solo a escribir código, sino también a generar ejemplos y explicaciones, como en el caso de Copilot.
- **Lenguajes de programación.** Es importante que el conjunto de datos abarque una variedad de lenguajes de programación para que el modelo sea flexible. Los lenguajes populares como Python, JavaScript, Java y C++ suelen ser prioritarios, pero es beneficioso incluir lenguajes menos comunes (por ejemplo, Rust y Fortran) para asegurar la adaptabilidad del modelo a diferentes contextos de programación.
  - Por ejemplo, la Figura 5 muestra la distribución de lenguajes de programación en los datos de entrenamiento en los modelos del repositorio CSDMS *(community surface dynamics modeling system)*.

© Universidad Internacional de La Rioja (UNIR)

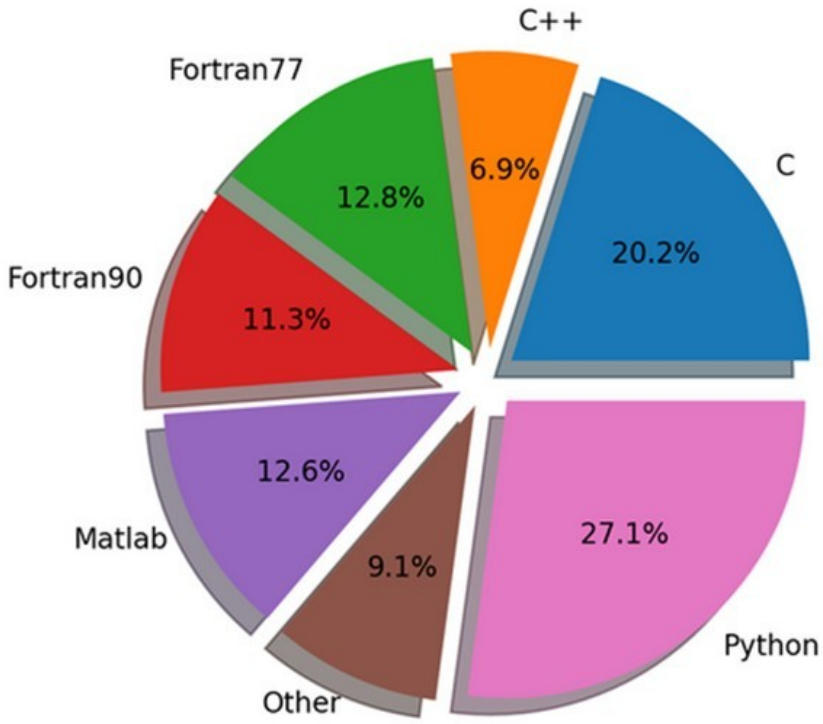


Figura 5. Distribución de lenguajes de programación en el repositorio de modelos CSDMS. Fuente: Hutton et al., 2022.

Preprocesamiento de datos

El preprocesamiento de los datos de código es un paso esencial para asegurar que el modelo pueda **interpretar y generar código** correctamente. Este proceso incluye diversas etapas que permiten **limpiar, normalizar y codificar** el código antes de utilizarlo para el entrenamiento.

© Universidad Internacional de La Rioja (UNIR)

Limpieza de datos

La limpieza de datos implica la eliminación de **código irrelevante o problemático,** como:

- Comentarios innecesarios. Eliminar comentarios que no aporten valor semántico al código.
- Código duplicado. Identificar y remover fragmentos de código duplicado, ya que esto puede sesgar el modelo hacia ciertos patrones.
- Errores sintácticos. Filtrar fragmentos de código que contengan errores de compilación o ejecución. Entrenar el modelo con código defectuoso puede degradar su rendimiento.

La Figura 6 muestra las diferentes etapas de la limpieza de datos, desde la recolección de datos hasta la eliminación de duplicados y la corrección de errores sintácticos.

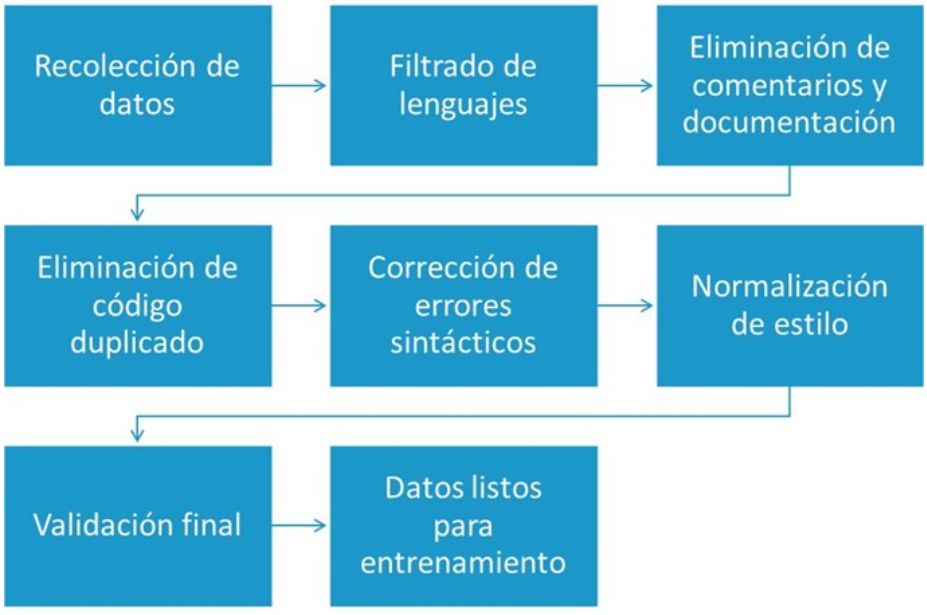


Figura 6. Etapas de la limpieza de datos. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Tokenización del código

Una de las fases más importantes en el preprocesamiento es la *tokenización.* El código se divide en ***tokens,*** que son las **unidades mínimas significativas,** como palabras clave, operadores, variables y estructuras de control. En este paso, se adapta el formato del **código** para que pueda ser interpretado por el modelo.

A diferencia de la *tokenización* de texto, que divide en palabras o subpalabras, la *tokenización* de código debe capturar correctamente las estructuras de control y los operadores lógicos.

> Un operador como `==` o una llave `{}` debe ser considerado un único *token.*

Codificación de *tokens*

Los *tokens* generados se codifican en **vectores numéricos** que pueden ser procesados por el modelo. Esta fase convierte a los *tokens* en **representaciones** que capturan la semántica del código, de manera que el modelo pueda entender las relaciones entre estos. Los enfoques como *byte-pair encoding* (BPE) son comunes para **manejar vocabularios grandes,** lo que reduce la cantidad de *tokens* únicos y mejora la eficiencia del modelo.

> Para una línea de código como `if (x > 10) { return x; }`, el modelo podría *tokenizarlo* en `['if', '(', 'x', '>', '10', ')', '{', 'return', 'x', ';', '}']`.

© Universidad Internacional de La Rioja (UNIR)

Normalización del código

La normalización del código implica aplicar **reglas uniformes de formato,** como indentación y espaciado, para que el código sea coherente y más fácil de procesar por el modelo. Además, se pueden **reemplazar nombres de variables y funciones** con identificadores genéricos (por ejemplo, `var1` , `func1`) para **reducir el ruido en los datos** y evitar sesgos hacia ciertos estilos de nombramiento.

Representación semántica

En algunos casos, es útil representar el código de manera semántica, por ejemplo, mediante **árboles de sintaxis abstracta** (AST).

Un AST representa una estructura sintáctica de un fragmento de código en un formato jerárquico que le facilita al modelo comprender la estructura del programa, más allá de los simples *tokens.*

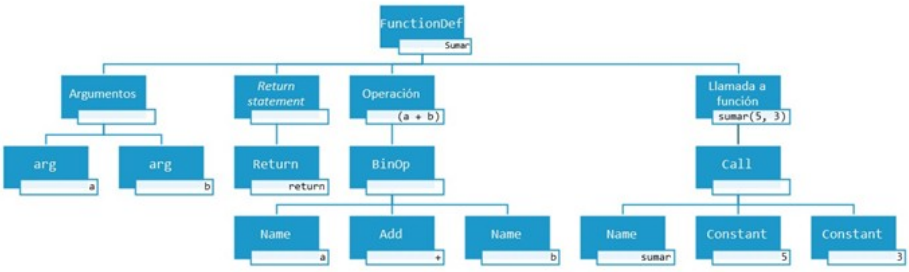


Figura 7. AST de una función sumar en Python. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

La Figura 7 muestra un árbol de sintaxis abstracta (AST) de cómo se representa una **función simple en Python** (sumar a+b), de manera que se divide la estructura en nodos que representan las **diferentes operaciones** y **llamadas a funciones.** A continuación, se muestra el código equivalente a dicho árbol:

```
def sumar(a, b):

    return a + b

resultado = sumar(5, 3)
```

Desafíos en la selección y preprocesamiento de datos

El preprocesamiento de datos para modelos generativos de código enfrenta varios desafíos:

- **Variedad en estilos de codificación.** Dado que los datos provienen de diferentes fuentes, los estilos de codificación pueden variar ampliamente, lo que dificulta la uniformidad en los datos de entrenamiento. Los modelos pueden verse influenciados por ciertas convenciones que no son estándar en todos los lenguajes o contextos.

- **Sesgos en los datos.** Al entrenar los modelos en repositorios de código público, es probable que ciertos patrones o estilos de codificación estén sobrerrepresentados. Por ejemplo, lenguajes populares como Python y JavaScript dominan los conjuntos de datos, lo que puede hacer que el modelo genere mejor el código en estos lenguajes y no sea tan preciso en otros menos comunes.

- **Control de calidad del código.** Es fundamental asegurar que el código utilizado sea de alta calidad, ya que entrenar el modelo con código defectuoso o mal estructurado podría producir recomendaciones incorrectas o poco eficientes.

© Universidad Internacional de La Rioja (UNIR)

Impacto del preprocesamiento en el rendimiento del modelo

El preprocesamiento adecuado de los datos **mejora significativamente el rendimiento** de los modelos generativos de código. Al proporcionar datos limpios, coherentes y bien estructurados, el modelo puede generalizar mejor, generar código más preciso y eficiente, y **adaptarse a una amplia variedad de lenguajes y estilos** de programación.

En un estudio reciente, se demostró que los modelos generativos de código entrenados con **datos bien preprocesados** generan soluciones de programación con **menos errores semánticos** y con **estructuras más legibles** para los humanos (Rozière et al., 2020).

© Universidad Internacional de La Rioja (UNIR)

1.4. Modelos de lenguaje especializados en programación

En los últimos años, los **modelos de lenguaje especializados en programación** revolucionaron el desarrollo de *software* y la automatización de tareas relacionadas con la **escritura,** la **corrección** y la **optimización de código.**

Los modelos de lenguaje especializados en programación utilizan **redes neuronales,** específicamente basadas en arquitecturas *transformer,* como GPT-3, Codex y PolyCoder. Estos modelos se entrenan para **comprender y generar código** en múltiples lenguajes.

Los *transformers,* gracias a su mecanismo de autoatención, permiten que el modelo entienda la relación entre diferentes elementos del código, como variables y funciones.

Además, estos modelos pueden manejar **grandes secuencias de código,** por lo que proporcionan **sugerencias contextuales** para completar bloques de código complejos. Sin embargo, presentan limitaciones, como la incap**acidad para generar código altamente eficiente** o manejar contextos extremadamente largos en proyectos grandes. PolyCoder, por ejemplo, demostró ser eficaz en lenguajes menos comunes, lo que amplió la capacidad de los modelos, más allá de lenguajes populares como Python o JavaScript, para incluir lenguajes científicos como Fortran o COBOL.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Introducción a los modelos de lenguaje especializados en programación

Los modelos de lenguaje son algoritmos **entrenados para predecir** la siguiente palabra o secuencia de *tokens* en base a un conjunto de entrada dado. En el caso de los modelos especializados en programación, estos están entrenados en **vastos repositorios de código** (como GitHub) para aprender la **sintaxis, semántica y patrones comunes** en múltiples lenguajes de programación.

Uno de los primeros experimentos que demostró la capacidad de los modelos de lenguaje para programar fue la **generación de código básico** en lenguajes como Python a partir de descripciones en lenguaje natural.

Principales modelos de lenguaje especializados en programación

A partir de los primeros experimentos para generar código en Python, empresas como OpenAI y DeepMind comenzaron a investigar **modelos específicos** para tareas de programación más complejas. Los principales modelos en la actualidad son las que se desarrollan a continuación.

OpenAI Codex

Codex es uno de los modelos más conocidos por su especialización en la **generación de código.** Derivado de GPT-3, Codex fue entrenado específicamente en datos de código fuente para comprender y generar código en más de una **docena de lenguajes de programación.** Su capacidad para interpretar descripciones en lenguaje natural y convertirlas en código funcional lo hace una herramienta poderosa para el **desarrollo asistido.**

© Universidad Internacional de La Rioja (UNIR)

Un usuario puede darle una instrucción como «escribe una función en Python que ordene una lista usando el algoritmo de ordenamiento burbuja» y Codex generará el siguiente código:

```
 def bubble_sort(arr):

n = len(arr)

for i in range(n):

for j in range(0, n-i-1):

if arr[j] > arr[j+1]:

arr[j], arr[j+1] = arr[j+1], arr[j]

 return arr
```

Codex también alimenta herramientas como GitHub Copilot, de manera que sugiere fragmentos de código, mientras que el desarrollador escribe, lo que acelera la productividad.

© Universidad Internacional de La Rioja (UNIR)

La Figura 8 muestra el flujo de entrenamiento de Codex: cómo los datos de código fuente se preprocesan, se introducen en la arquitectura del modelo y producen resultados en forma de código.



Figura 8. Flujo de entrenamiento de un modelo Codex. Fuente: elaboración propia.

DeepMind AlphaCode

AlphaCode, desarrollado por DeepMind, es otro modelo que se especializa en la generación de código. A diferencia de Codex, AlphaCode está más centrado en la **resolución de problemas algorítmicos.** Fue entrenado no solo para generar código, sino también para **abordar desafíos** de programación competitiva. Este tipo de modelo puede resolver problemas complejos que requieren más que simples conocimientos de sintaxis, de manera que emplea **soluciones algorítmicas avanzadas.**

© Universidad Internacional de La Rioja (UNIR)

AlphaCode logró participar en competiciones de programación y resolver problemas a un nivel cercano al de programadores humanos de élite.

La Figura 9 muestra las principales diferencias entre Codex y AlphaCode en cuanto a su arquitectura, datos de entrenamiento y aplicaciones típicas.

Aspecto	Codex	AlphaCode
Arquitectura	GPT-3	*Transformers* optimizados para algoritmo complejos
Datos	GitHub	Competiciones de programación
Lenguajes	Python, Javascript y Ruby	Python, C++ y Java
Aplicaciones	Generación de código	Asistencia en algoritmos complejos
Optimización	En tiempo real	Enfocada en algoritmia
Interacción	Asistencia en IDE, sugerencias y traducción	Resolución sin intervención humana y *benchmarking*
Casos de uso	Generación de funciones y autocompletado	Resolución automatizada y participación en competiciones

Tabla 1. Diferencias entre Codex y AlphaCode. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

PolyCoder

PolyCoder es otro modelo que se destaca por su especialización en **múltiples lenguajes de programación.** Fue entrenado para generar código en varios lenguajes simultáneamente, por lo que supera a otros modelos en lenguajes menos populares. PolyCoder emplea técnicas de **transferencia de conocimiento** entre lenguajes, lo que le permite **generar un código más preciso** en lenguajes con menos datos disponibles.

PolyCoder puede generar código en lenguajes como **Fortran,** que, a menudo, es utilizado en la computación científica y que cuenta con menos datos de entrenamiento en comparación con lenguajes más modernos como Python o JavaScript.

Arquitecturas de los modelos de lenguaje para programación

Los modelos de lenguaje especializados en programación se basan principalmente en la **arquitectura *transformer,*** introducida por Vaswani et al. (2017), que permite el procesamiento eficiente de **secuencias largas,** como el código fuente.

La Figura 9 muestra la arquitectura *transformer* aplicada a la generación de código, por lo que muestra las fases de **codificación, atención y decodificación.**

© Universidad Internacional de La Rioja (UNIR)

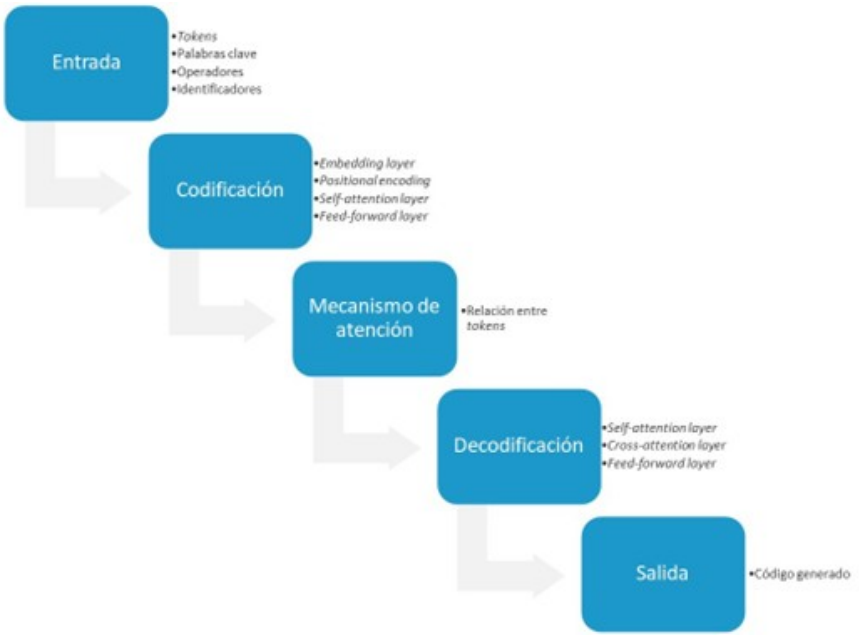


Figura 9. Arquitectura *transformer* aplicada a la generación de código. Fuente: elaboración propia.

Algunos modelos avanzados de lenguaje para programación implementaron técnicas de *meta-learning* para **aprender a generar código** a partir de muy pocos ejemplos, lo que facilita su **aplicación en lenguajes menos documentados** o con poca información disponible.

Futuras direcciones en los modelos de lenguaje especializados en programación

Los avances futuros en estos modelos se enfocarán en:

- Mayor contextualización. Mejorar la capacidad del modelo para comprender el contexto global de un proyecto de *software* completo, no solo de fragmentos aislados.
- Optimización del rendimiento. Desarrollar modelos que no solo generen código correcto, sino que optimicen el rendimiento, como reducir la complejidad algorítmica.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- Generación de documentación. Automatización completa de la generación de documentación técnica basada en el código, lo que reduce aún más el trabajo manual de los desarrolladores.

© Universidad Internacional de La Rioja (UNIR)

1.5. Evaluación de modelos de generación de código

La evaluación de los modelos de generación de código es una etapa crucial para **medir su efectividad y calidad.** Dado que estos modelos generan código automáticamente, es importante verificar que los fragmentos de código producidos no solo sean **sintácticamente correctos,** sino también **funcionales, eficientes y coherentes** con las expectativas del problema.

Evaluar los modelos de generación de código implica el uso de **métricas específicas** como BLEU, que mide la similitud entre el código generado y el código de referencia, y *pass@k,* que mide cuántos intentos correctos realiza el modelo. Sin embargo, estas métricas **no son perfectas,** ya que un código que pasa las pruebas automáticas puede **no ser óptimo** o estar mal estructurado.

Un enfoque más avanzado es utilizar métricas como la **complejidad ciclomática,** que evalúa el número de rutas posibles en el código generado para asegurar que el código sea lo más **sencillo y eficiente posible.** Además de las evaluaciones automáticas, las **pruebas manuales** son esenciales para detectar errores lógicos que el modelo no puede identificar.

La evaluación de modelos de generación de código es un proceso complejo que implica la evaluación de la funcionalidad, la eficiencia, la legibilidad y la adaptabilidad a diferentes contextos.

En el contexto empresarial, la evaluación de modelos se complementa con la **integración continua en *pipelines*** de CI y CD, donde el código generado es automáticamente **probado y evaluado** antes de ser desplegado.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

A medida que los modelos de generación de código siguen evolucionando, también lo harán las **técnicas** y las **herramientas** para evaluar su desempeño de manera más precisa y eficiente. A continuación, se detallan las diferentes **métricas, enfoques y desafíos asociados** a la evaluación de estos modelos.

Métricas de evaluación

Exactitud sintáctica

La exactitud sintáctica mide si el código generado es **gramaticalmente correcto** en términos de las reglas del lenguaje de programación utilizado. Este es el primer paso en la evaluación, ya que un código que no se puede compilar o interpretar es de poca utilidad.

> Si el modelo genera:
>
> ```
> if (x > 10) {
> return x;
> ```
>
> en lugar de
>
> ```
> if (x > 10) {
> return x;
> }
> ```
>
> La falta de la llave de cierre } indicaría un error de sintaxis.

© Universidad Internacional de La Rioja (UNIR)

Exactitud semántica

Incluso si el código es sintácticamente correcto, es fundamental que cumpla con la **lógica esperada.** La exactitud semántica evalúa si el código hace lo que se supone que debe hacer según la especificación o el enunciado del problema.

> Un modelo genera código para calcular la suma de una lista de números. Si bien el código puede ser válido en cuanto a sintaxis, puede multiplicar los números en lugar de sumarlos, por lo que no cumpliría con la semántica deseada.

Funcionalidad y correctitud

La funcionalidad se refiere a si el código generado produce el **resultado esperado** cuando se ejecuta. En muchos casos, los modelos son evaluados al ejecutar el código en diversos entornos de prueba con **entradas específicas** y al verificar si las **salidas coinciden** con las esperadas. Por ejemplo, en competiciones como Codeforces o LeetCode, los modelos como AlphaCode se evalúan al resolver problemas reales de programación y al verificar si las soluciones son correctas para todas las entradas de prueba.

Métrica CodeBLEU *(bilingual evaluation understudy)*

CodeBLEU es una métrica diseñada específicamente para **evaluar la calidad** de la generación de código. A diferencia de otras métricas como BLEU (que se utiliza comúnmente en traducción automática y generación de texto), CodeBLEU tiene en cuenta las **particularidades del código de programación,** como la estructura, la sintaxis y la semántica.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

El principal objetivo de CodeBLEU es proporcionar una evaluación más precisa y relevante para tareas de generación de código en comparación con BLEU, que solo considera la similitud superficial de secuencias.

Imagina que un modelo genera un código para sumar dos números en Python. El código generado puede ser:

```
def sumar(a, b):

return a + b
```

El código de referencia puede ser:

```
def suma(a, b):

return b + a
```

Ambos fragmentos de código son semánticamente idénticos (producen el mismo resultado), pero, si solo usamos BLEU, la evaluación sería baja porque las secuencias de *tokens* difieren (`sumar` vs. `suma` y `a + b` vs. `b + a`). Sin embargo, CodeBLEU toma en cuenta que los dos códigos son equivalentes en su funcionalidad, lo que permite una evaluación más precisa.

© Universidad Internacional de La Rioja (UNIR)

La Figura 10 muestra el proceso para evaluar el código de acuerdo con la métrica CodeBLEU.

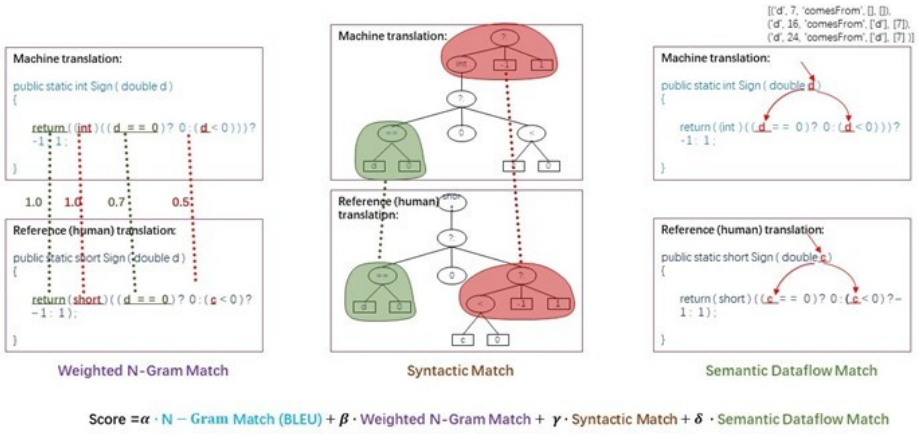


Figura 10. Proceso de evaluar código mediante CodeBLEU. Fuente: celbree, s. f.

Exactitud en la solución *(pass@k)*

La métrica *pass@k* mide **cuántas veces el código generado es correcto** dentro de las primeras k soluciones generadas por el modelo. Esta métrica es importante en modelos donde se generan **varias soluciones** para un problema. El *pass@k* es útil en entornos donde los desarrolladores pueden explorar varias soluciones propuestas y elegir la que **mejor se ajuste** a sus necesidades.

> Si un modelo genera cinco soluciones para un problema y, al menos, una es correcta, el *pass@k* será considerado exitoso. Este enfoque se utiliza en modelos como Codex.

© Universidad Internacional de La Rioja (UNIR)

Evaluación automática vs. manual

Evaluación automática

La evaluación automática es **rápida y escalable** y, generalmente, involucra la ejecución del código generado en pruebas automatizadas o el cálculo de métricas como BLEU o *pass@k.* Aunque eficiente, esta evaluación puede **no capturar completamente la calidad del código,** especialmente en términos de legibilidad y eficiencia.

La ventaja es que la evaluación automática permite evaluar grandes volúmenes de código en poco tiempo, lo que es fundamental para entrenar y ajustar modelos a gran escala.

Evaluación manual

La evaluación manual, llevada a cabo por programadores expertos, es más precisa en ciertos aspectos, como la **legibilidad del código** y la **calidad de la solución,** pero es mucho más costosa y lenta. Esta evaluación se enfoca en:

- Legibilidad. ¿Es el código fácil de leer y seguir? Un código generado puede ser funcional, pero ilegible, lo que lo haría difícil de mantener.
- Eficiencia. ¿El código es eficiente en términos de tiempo de ejecución y uso de memoria? Un código que cumple con los requisitos de funcionalidad podría ser ineficiente, lo que afecta al rendimiento del sistema en proyectos más grandes.

© Universidad Internacional de La Rioja (UNIR)

Evaluación en diferentes contextos

Problemas algorítmicos

En contextos algorítmicos, como la resolución de problemas en plataformas de competencia, la evaluación se enfoca principalmente en la **funcionalidad** y el **rendimiento.** Un código que cumple con las **restricciones del problema** en términos de tiempo y espacio, y pasa todas las pruebas se considera exitoso.

> El modelo puede ser evaluado en problemas de algoritmos de búsqueda, ordenación o cálculos numéricos, ya que se puede medir fácilmente si la solución es correcta y eficiente.

Generación de código para aplicaciones reales

En la generación de código para **aplicaciones reales,** como el desarrollo de aplicaciones web o sistemas embebidos, la evaluación va más allá de la funcionalidad. Se considera la calidad del código en términos de **modularidad, extensibilidad y compatibilidad** con los estándares del proyecto.

Traducción entre lenguajes de programación

La traducción entre lenguajes (por ejemplo, de Python a Java) es otro tipo de evaluación donde el código debe mantener su **exactitud semántica** al cambiar de un lenguaje a otro. Aquí, se evalúa si el código traducido sigue la **lógica del original** y si está **adaptado a las particularidades del lenguaje de destino.**

> Si el código original es una función en Python que utiliza listas dinámicas, el código traducido a C++ debería utilizar estructuras de datos adecuadas como `std::vector` .

© Universidad Internacional de La Rioja (UNIR)

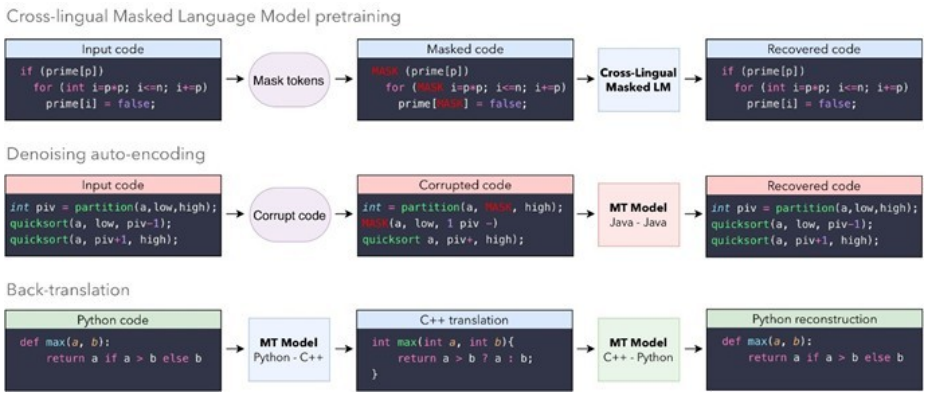


Figura 11. Principios seguidos en CodeGen para la generación de un código. Fuente: Rozière et al., 2020.

La Figura 11 muestra los principios seguidos en el modelo CodeGen (Rozière et al., 2020) para generación de código.

Desafíos en la evaluación de modelos generativos de código

Ambigüedad en los requisitos

A menudo, los problemas de programación pueden tener **múltiples soluciones correctas** y un código puede ser válido, aunque no coincida exactamente con el de referencia. Evaluar modelos en este contexto requiere **flexibilidad** para aceptar diferentes soluciones igualmente válidas.

Complejidad computacional

Algunos modelos generativos requieren **mucho tiempo y recursos** para generar y evaluar soluciones. Esto plantea el desafío de **equilibrar** la precisión de la evaluación con la eficiencia computacional.

© Universidad Internacional de La Rioja (UNIR)

Generalización

Un modelo que fue entrenado en un conjunto de datos específico puede **no generalizar bien** a nuevos dominios de programación o lenguajes no vistos. Evaluar cómo un modelo se adapta a diferentes tipos de problemas y contextos es crucial para **medir su versatilidad.**

Herramientas y plataformas de evaluación

Existen varias plataformas y herramientas que facilitan la evaluación de modelos de generación de código:

- Codex y GitHub Copilot. Estas herramientas cuentan con sistemas automáticos para evaluar la calidad del código generado en tiempo real, por lo que ejecutan pruebas y ofrecen múltiples sugerencias a los desarrolladores.
- Codeforces. En competiciones de programación, el código generado se evalúa automáticamente contra conjuntos de pruebas diseñados para evaluar tanto la exactitud como la eficiencia de las soluciones.

© Universidad Internacional de La Rioja (UNIR)

1.6. Casos de uso prácticos en desarrollo de software

El uso del AA en la generación de código está transformando diversas áreas del desarrollo de *software,* desde la automatización de tareas repetitivas hasta la creación de nuevas funcionalidades.

Los modelos de generación de código mediante AA no solo mejoran la productividad, sino que también reducen los errores humanos y aumentan la eficiencia general del desarrollo.

Las **aplicaciones prácticas** de la generación automática de código son diversas y abarcan desde **pequeños módulos** hasta **sistemas completos.** Por ejemplo, en el desarrollo de aplicaciones móviles, los desarrolladores pueden usar IA para generar automáticamente módulos de autenticación, interfaces de usuario y funciones de *backend* a partir de plantillas predefinidas o descripciones en lenguaje natural.

Otro caso práctico es la **integración de microservicios** en aplicaciones web, donde la IA puede generar API *(application programming interface)* RESTful (transferencias de estado representacionales) que interactúan con bases de datos y otros servicios, por lo que reducen el tiempo de desarrollo significativamente. Además, en el área de pruebas automáticas, las herramientas impulsadas por IA generan ***suites*** **de pruebas unitarias y de regresión,** lo que asegura que el código cumple con los requisitos funcionales antes de ser desplegado.

© Universidad Internacional de La Rioja (UNIR)

En entornos de desarrollo continuo (DevOps), la IA se integra en flujos de CI y CD para **validar automáticamente** los cambios en el código, ejecutar pruebas y realizar despliegues sin intervención manual, lo que **mejora la eficiencia operativa** y **reduce errores.** A continuación, se presentan algunos de los casos de uso más relevantes y prácticos en la industria actual.

Asistentes de codificación automatizada

GitHub Copilot

GitHub Copilot es una **herramienta** impulsada por modelos de lenguaje como Codex que asiste a los desarrolladores, por lo que **sugiere líneas o bloques de código** basados en el contexto del código que están escribiendo. La herramienta ayuda a mejorar la **productividad del desarrollador** al autocompletar funciones, sugerir algoritmos completos o incluso generar archivos de código entero.

> Un desarrollador que esté implementando una API REST en Python puede recibir sugerencias automáticas de rutas, controladores y validaciones según el patrón de desarrollo seguido. GitHub Copilot aprende del código circundante y sugiere piezas que se ajusten a la lógica esperada.

© Universidad Internacional de La Rioja (UNIR)

La Figura 12 muestra el *framework* entre el modelo GitHub Copilot y el editor de código, desde la escritura inicial del código hasta la generación de sugerencias automáticas.



Figura 12. *Framework* de GitHub Copilot. Fuente: García Q., 2021.

Tabnine

Tabnine es otro ejemplo de asistente basado en IA que permite generar código autocompletado, pero también tiene la capacidad de aprender del código específico de un equipo de desarrollo.

Esto lo hace muy valioso en proyectos grandes donde los patrones de diseño y las convenciones de codificación son clave. También puede **personalizar sus sugerencias** basándose en el código almacenado en un repositorio privado de la empresa, lo que permite que las recomendaciones sean más **precisas y ajustadas** a las reglas internas del equipo.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Generación de código boilerplate

El código boilerplate se refiere a las partes de código **repetitivas y necesarias** para configurar entornos o realizar tareas comunes en el desarrollo de *software.* Algunos modelos de generación de código pueden **automatizar la creación** de este tipo de código, de manera que les permiten a los desarrolladores centrarse en las partes más importantes y personalizadas de la aplicación.

Frameworks backend

Al desarrollar API o **sistemas *backend,*** gran parte del código está destinado a **configuraciones iniciales,** autenticación, manejo de bases de datos, etc. Además, algunas herramientas impulsadas por modelos generativos pueden crear automáticamente este código repetitivo basado en **plantillas predefinidas** o especificaciones de alto nivel.

Un desarrollador puede solicitarle a un modelo generativo la creación de una **estructura básica** de un servicio RESTful con autenticación JWT en Node.js. El modelo generará el código necesario para **configurar rutas, *middleware* y validaciones.**

Herramientas como Yeoman o JHipster permiten la creación de plantillas y *scaffolding* para aplicaciones y pueden ser integradas con modelos de IA para mejorar la generación automática de código.

© Universidad Internacional de La Rioja (UNIR)

Generación de código multiplataforma

Uno de los desafíos en el desarrollo de *software* moderno es la necesidad de **soportar múltiples plataformas,** como aplicaciones móviles (iOS y Android), web y escritorio. Algunos modelos de IA pueden **automatizar la conversión de código** entre plataformas, lo que reduce drásticamente el tiempo de desarrollo.

Traducción entre lenguajes de programación

Algunos modelos generativos están entrenados en mú**ltiples lenguajes de programación,** por lo que pueden traducir código entre estos lenguajes. Esto es particularmente valioso para empresas que necesitan **portar aplicaciones de una plataforma a otra** sin tener que reescribir todo el código manualmente. Por ejemplo, un modelo puede traducir código escrito en Swift, para una aplicación iOS, a Kotlin, para una aplicación Android. De esta forma, los desarrolladores no necesitan reescribir manualmente toda la lógica, sino solo ajustar aspectos específicos del sistema operativo.

Algunos modelos pueden mantener no solo la **lógica del programa,** sino también las **convenciones estilísticas de cada lenguaje,** lo que genera un código que es más legible y mantenible para los desarrolladores.

Generación automática de pruebas unitarias y de integración

El **desarrollo de pruebas** es un componente clave en cualquier flujo de trabajo de desarrollo ágil. La creación de pruebas automáticas puede **consumir mucho tiempo,** especialmente en proyectos grandes. De esta manera, los modelos generativos pueden ayudar a los desarrolladores a **escribir pruebas automáticas,** como las pruebas unitarias y de integración, basadas en el código que ya ha sido escrito.

© Universidad Internacional de La Rioja (UNIR)

Pruebas unitarias

Herramientas como *diffblue cover* utilizan IA para generar automáticamente pruebas unitarias en lenguajes como Java.

Por tanto, analizan el código fuente, identifican la funcionalidad y crean pruebas que cubren las principales ramas y condiciones de ejecución.

Por ejemplo, un modelo de IA puede generar automáticamente una prueba unitaria para verificar que una función de cálculo de impuestos devuelva el valor correcto para una variedad de entradas.

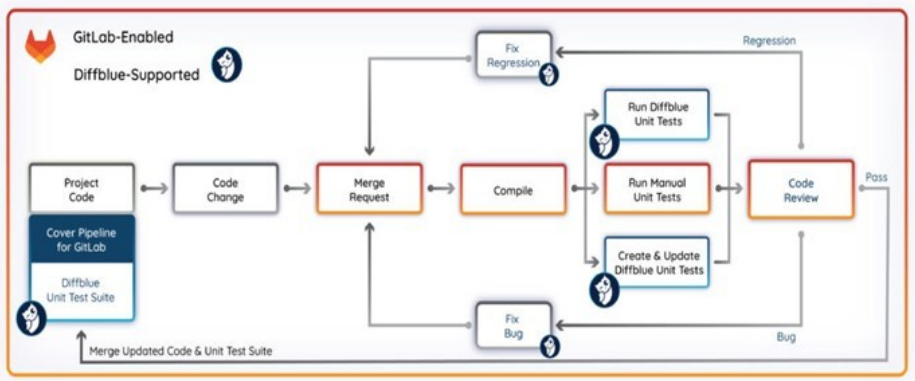


Figura 13. Esquema de integración de *diffblue cover* en el *pipeline* de un proyecto GitLab. Fuente: Diffblue.com

La Figura 13 muestra el esquema de integración de *diffblue cover* en el *pipeline* de un proyecto en GitLab.

© Universidad Internacional de La Rioja (UNIR)

Pruebas de integración

Además de las pruebas unitarias, algunos modelos son capaces de generar **pruebas de integración** que verifican que diferentes módulos de una aplicación **interactúan correctamente.** Por ejemplo, en una aplicación de *e-commerce,* un modelo de IA podría generar pruebas de integración para verificar que el módulo de procesamiento de pagos interactúe correctamente con el módulo de gestión de inventario.

Optimización de código con aprendizaje automático

Los modelos generativos no solo pueden escribir código, sino también optimizarlo para mejorar su eficiencia.

Algunas herramientas basadas en IA pueden sugerir formas de **refactorizar código, eliminar redundancias** o **mejorar el rendimiento** general de la aplicación.

Refactorización automática

La refactorización es el proceso de **reestructurar el código existente** sin cambiar su funcionalidad. Modelos como Facebook Aroma pueden analizar el código y sugerir patrones de refactorización que **mejoren la legibilidad y mantenibilidad del código.** Por ejemplo, si el código contiene funciones que pueden ser simplificadas o combinadas, el modelo sugerirá un refactor para reducir la duplicación de código.

Este tipo de optimización no solo ayuda a reducir errores, sino también a **mejorar la eficiencia** del equipo de desarrollo al evitar la acumulación de deuda técnica.

© Universidad Internacional de La Rioja (UNIR)

Optimización del rendimiento

Algunos modelos de AA también pueden **optimizar el código** para mejorar el rendimiento en áreas clave como el **uso de memoria** o el tiempo de ejecución. Por ejemplo, en un algoritmo de ordenamiento implementado en C++, un modelo podría sugerir una versión más eficiente del algoritmo basada en un análisis del tiempo de ejecución para diferentes tamaños de entrada.

Modelos como AlphaCode, especializados en problemas algorítmicos complejos, pueden desarrollar más **códigos óptimos** que la mayoría de los programadores promedio.

Generación automática de documentación

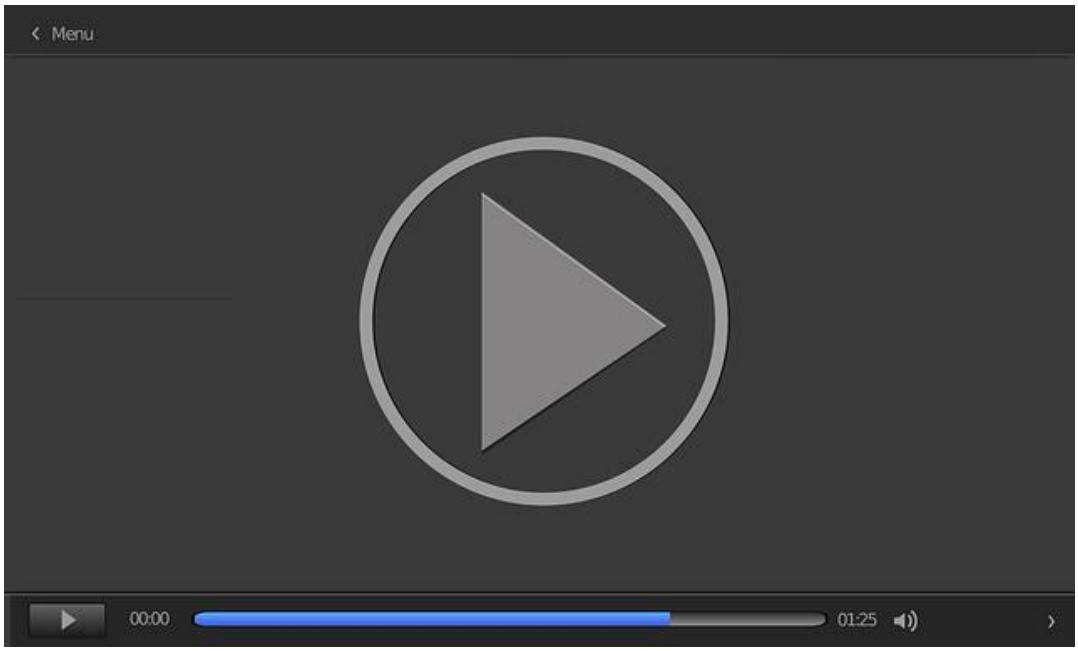
La **documentación** es una parte esencial del desarrollo de *software,* pero también una de las más tediosas. Algunos modelos de IA pueden generar automáticamente documentación a partir del código, de manera que describen su **funcionalidad, argumentos y comportamientos esperados.** Por ejemplo, al escribir una función en Python, el modelo puede generar automáticamente un *docstring* que describe los parámetros de entrada, el valor de retorno y las posibles excepciones.

Algunos modelos de IA son capaces de generar **comentarios en varios idiomas,** lo que facilita la colaboración en equipos internacionales.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

A continuación, vamos a ver el vídeo *Aprendizaje por refuerzo para la Generación de Código.* Este vídeo introduce el concepto de aprendizaje por refuerzo aplicado a la generación de código, un enfoque complementario a los modelos supervisados tradicionales. Se explicará cómo los agentes pueden ser entrenados para generar código mediante recompensas, por lo que mostraremos ejemplos prácticos en los que este tipo de aprendizaje mejora la eficiencia en la solución de problemas. Se incluirá una explicación de los algoritmos clave (como *Q-learning* o *deep Q-networks*) y su implementación en entornos de desarrollo.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=e88e6334-1589-4c1d-854c-b21400db625b

© Universidad Internacional de La Rioja (UNIR)

1.7. Referencias bibliográficas

Allamanis, M., Barr, E. T., Devanbu, P. y Sutton, C. (2018). A Survey of Machine Learning for Big Code and Naturalness. *ACM Computing Surveys (CSUR), 51*(4), pp. 1-37.

Celbree. (s. f.). *CodeXGLUE.* https://github.com/microsoft/CodeXGLUE

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S.,… Zaremba, W. (2021). Evaluating Large Language Models Trained on Code. *Arxiv.* https://arxiv.org/abs/2107.03374

Fraser, G. y Arcuri, A. (2012). Whole test suite generation. *IEEE Transactions on Software Engineering, 39*(2), pp. 276-291.

García Q., L. F. (2021, julio 1). Pair programmer with Copilot. *Medium.* https://lufgarciaqu.medium.com/pair-programmer-with-github-6ca825d87dcd

Hutton, E. W. H., Piper, M. D. y Tucker, G. E. (2022). The Babelizer: language interoperability for model coupling in the geosciences. *The Journal of Open Source Software,* *7*(71). https://www.researchgate.net/publication/359355897_The_Babelizer_language_interoperability_for_model_coupling_in_the_geosciences

Ray, B., Hellendoorn, V. J., Godhane, S., Tu, Z., Bacchelli, A. y Devanbu, P. (2016). *On the "Naturalness" of Buggy Code* [presentación en conferencia]. 2016 IEEE/ACM 38th International Conference on Software Engineering, Texas, Estados Unidos. https://dl.acm.org/doi/10.1145/2884781.2884848

© Universidad Internacional de La Rioja (UNIR)

Rozière, B., Lachaux, M. A., Chanussot, L. y Lample, G. (2020). *Unsupervised Translation of Programming Languages* [presentación en conferencia]. NIPS'20: Proceedings of the 34th International Conference on Neural Information Processing Systems, Nueva York, Estados Unidos de América.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. (2017). *Attention is all you need. Advances in Neural Information Processing Systems.* Curran Associates, Inc.

Xu, F., Vasilescu, B. y Neubig, G. (2022). In-IDE Code Generation from Natural Language: Promise and Challenges. *ACM Transactions on Software Engineering and Methodology, 31*(2), pp. 1-47. https://doi.org/10.1145/3487569

© Universidad Internacional de La Rioja (UNIR)

GitHub Copilot: the AI-powered code completion tool

GitHub. (s. f.). *Documentación GitHub Copilot.* https://docs.github.com/es/copilot

Un tutorial exhaustivo sobre cómo usar GitHub Copilot, que aplica modelos de lenguaje como Codex para sugerir código, mientras que el desarrollador trabaja. Este recurso es clave para aprender a aprovechar la IA en el flujo de trabajo diario de un desarrollador.

© Universidad Internacional de La Rioja (UNIR)

OpenAI Codex: evaluating large language models trained on code

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S.,… Zaremba, W. (2021). Evaluating Large Language Models Trained on Code. *Arxiv.* https://arxiv.org/abs/2107.03374

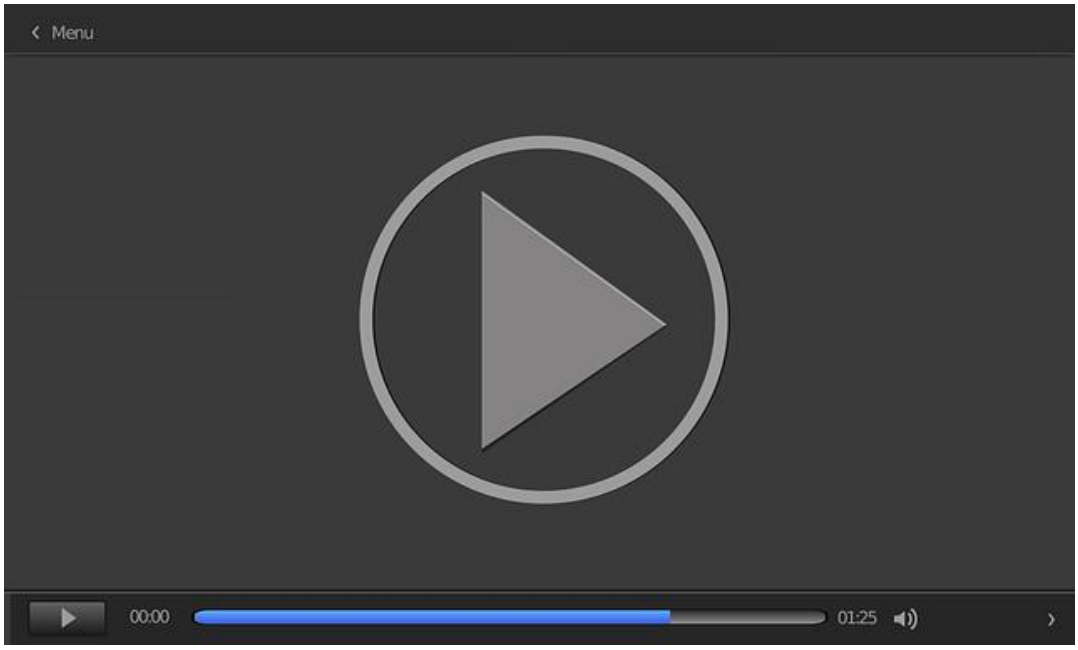
Un artículo que analiza cómo OpenAI Codex fue entrenado para generar código en varios lenguajes de programación. Además, se discuten los desafíos de generar código efectivo y cómo este modelo mejora la productividad.

© Universidad Internacional de La Rioja (UNIR)

Machine learning for code generation (vídeo)

Two Minute Papers. (2022, marzo 17). *DeepMind's New AI: As Smart As An Engineer... Kind Of!* [Vídeo]. YouTube. https://www.youtube.com/watch?v=x_cxDgR1x-c

Vídeo explicativo sobre cómo el aprendizaje automático revolucionó la generación de código. Detalla los avances en modelos como AlphaCode y su aplicación en entornos de desarrollo.



Accede al vídeo:

https://www.youtube.com/embed/x_cxDgR1x-c?si=ASPZY28AU9iGzr21

© Universidad Internacional de La Rioja (UNIR)

Aroma: using machine learning for code recommendation

Facebook AI Research. (2020). *Aroma: Using machine learning for code recommendation.* https://ai.meta.com/blog/aroma-ml-for-code-recommendation/

Artículo que detalla cómo Facebook Aroma utiliza IA para la recomendación automática de código. Este recurso es esencial para entender cómo se puede mejorar la legibilidad y la eficiencia del código mediante IA.

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Cuál es uno de los usos principales del AA en la generación de código?

A. Elaborar gráficos visuales.

B. Generar código a partir de descripciones en lenguaje natural.

C. Crear *hardware* de alta precisión.

D. Rediseñar redes neuronales.

2. ¿Qué herramienta utiliza modelos de IA para sugerir código automáticamente en entornos de desarrollo?

A. Photoshop.

B. GitHub Copilot.

C. Microsoft Word.

D. AutoCAD.

3. ¿Cuál es el objetivo de los modelos generativos aplicados a la programación?

A. Mejorar el rendimiento del *hardware.*

B. Automatizar la generación y la corrección de código.

C. Crear interfaces gráficas avanzadas.

D. Traducir textos en tiempo real.

4. ¿Qué proceso permite que el código se reestructure para mejorar su mantenibilidad sin cambiar su funcionalidad?

A. La traducción de código.

B. La refactorización automática.

C. La depuración manual.

D. El diseño de *hardware.*

© Universidad Internacional de La Rioja (UNIR)

5. ¿Qué modelo es reconocido por su capacidad para traducir entre lenguajes de programación?

A. GitHub.

B. TransCoder.

C. PowerPoint.

D. TensorFlow.

6. ¿Qué técnica se utiliza para dividir el código en unidades mínimas significativas durante el preprocesamiento?

A. La normalización.

B. La *tokenización*.

C. La fragmentación.

D. La compilación.

7. ¿Qué métrica se utiliza para evaluar la coincidencia entre el código generado y el de referencia?

A. BLEU.

B. El tiempo de ejecución.

C. La *CPU utilization.*

D. La capacidad de memoria.

8. ¿Cuál es uno de los desafíos de la generación automática de código?

A. Generar gráficos en 3D.

B. Asegurar la calidad y la funcionalidad del código generado.

C. Crear páginas web.

D. Diseñar algoritmos de alta precisión.

© Universidad Internacional de La Rioja (UNIR)

9. ¿Qué técnica les permite a los modelos de lenguaje mejorar continuamente a partir de interacciones previas?

A. La optimización Bayesiana.

B. El aprendizaje por refuerzo.

C. La búsqueda aleatoria.

D. La prueba de Turing.

10. ¿Qué representa el modelo Codex en el contexto del AA aplicado a programación?

A. Una herramienta de diseño gráfico.

B. Un modelo de lenguaje especializado en generación de código.

C. Un sistema operativo.

D. Un programa de edición de vídeo.

11. ¿Qué significa preprocesamiento de datos en la generación de código con IA?

A. Compilar el código.

B. Limpiar, *tokenizar* y codificar el código fuente.

C. Crear *hardware* nuevo.

D. Mejorar el rendimiento gráfico.

12. ¿Qué herramienta ayuda a los desarrolladores a escribir pruebas automáticas mediante IA?

A. Diffblue Cover.

B. Notepad.

C. Excel.

D. AutoML.

© Universidad Internacional de La Rioja (UNIR)

13. ¿Cuál es una de las limitaciones actuales de los modelos generativos de código?

A. La alta eficiencia computacional.

B. Su tendencia a cometer errores en lógica compleja.

C. La automatización de tareas administrativas.

D. Facilitar la creación de videojuegos.

14. ¿Qué componente en los modelos de lenguaje permite la generación de código en varios lenguajes?

A. La arquitectura *transformer*.

B. El compilador de *hardware.*

C. La red neuronal de baja capacidad.

D. Las herramientas ofimáticas.

15. ¿Cuál es una de las ventajas de las herramientas de autocompletado como GitHub Copilot?

A. Automatiza las pruebas gráficas.

B. Mejora la velocidad de escritura de código.

C. Aumenta el tamaño del código.

D. Facilita la creación de contenido multimedia.

16. ¿Qué tipo de datos es necesario para entrenar los modelos de lenguaje especializados en programación?

A. Imágenes de alta resolución.

B. Grandes volúmenes de código fuente.

C. Audio y vídeo.

D. Datos financieros.

© Universidad Internacional de La Rioja (UNIR)

17. ¿Como funciona la métrica pass@k?

A. Comprueba si el código ejecutado pasa las pruebas unitarias.

B. Evalúa cuántas veces el código generado es correcto.

C. Somete el código generado a pruebas de estrés.

D. Mide el número de pruebas unitarias superadas por el código generado.

18. ¿Cuál es uno de los modelos más avanzados en la traducción automática de código entre lenguajes?

A. AlphaCode.

B. PolyCoder.

C. TensorFlow.

D. JavaScript.

19. ¿Qué permite la generación automática de documentación en código fuente?

A. Aumentar la complejidad del código.

B. Facilitar la comprensión del código generado.

C. Crear nuevos lenguajes de programación.

D. Reducir el tamaño del código.

20. ¿Qué técnica se usa para optimizar el rendimiento de un algoritmo sin cambiar su funcionalidad?

A. La refactorización.

B. El *debugging.*

C. La búsqueda aleatoria.

D. La reescritura manual.

© Universidad Internacional de La Rioja (UNIR)